Sage: Safe and Adaptive In-Kernel Neural Inference via eBPF for Real-Time Network Intrusion Detection

Anonymous
Anonymous Institution
anon@example.com

ABSTRACT

Deploying machine learning models inside the Linux kernel via eBPF/XDP promises microsecond-level network intrusion detection, but the eBPF verifier imposes severe constraints: no floating-point arithmetic, a 512-byte stack, and a one-million-instruction limit. Existing approaches either suffer from excessive quantization error (int8), poor memory efficiency (decision trees), or lack rigorous analysis of arithmetic safety (int32).

We present Sage, a system for *safe* and *adaptive* in-kernel neural network inference. Sage makes three contributions: (1) A *formal overflow-safety analysis* that derives, for any MLP architecture and enlargement factor $s = 2^b$, a closed-form bound on the maximum safe b, resolving an open question left by prior empirical approaches. (2) An *automated design-space exploration framework*, inspired by the autoresearch methodology, that systematically sweeps 48 architecture configurations and identifies Pareto-optimal trade-offs between accuracy, BPF instruction count, and memory. (3) A complete *four-stage tail-call inference engine* with per-CPU scratch maps, evaluated on a 200K-sample imbalanced intrusion-detection dataset across four attack classes.

Our exploration reveals that b=16 is a critical quantization threshold (below which accuracy degrades by 10–30%), that hidden-layer width saturates at h=8 for this task (99.6% at just 656 B), and that the theoretical overflow bound is 5,590× more conservative than empirical reality. The best configuration achieves 99.6% accuracy with only 754 BPF instructions and 1.8 KB model memory, requiring no floating-point support.

CCS CONCEPTS

• **Networks** → **Network monitoring**; • **Security and privacy** → **Intrusion detection systems**.

KEYWORDS

eBPF, XDP, in-kernel machine learning, quantized inference, intrusion detection

1 INTRODUCTION

Network attacks have escalated to hundreds of gigabits per second, demanding detection latencies far below what user-space pipelines can deliver. The eBPF/XDP subsystem in the Linux kernel offers a compelling alternative: programs attached at the XDP hook process packets before they enter the network stack, achieving single-digit microsecond latency [7].

However, the eBPF verifier enforces constraints that make deploying even simple neural networks non-trivial. Programs cannot use floating-point arithmetic, each program is limited to a 512-byte stack, the total instruction budget per program is one million, and no dynamic memory allocation is permitted. These constraints preclude standard ML inference and force a co-design of model quantization, program architecture, and memory layout.

Prior work has explored three families of approaches. *Decision trees* (DT) are structurally simple but memory-hungry and difficult to update online [1]. *Int8-quantized neural networks* achieve fast inference but suffer unacceptable precision loss [3]. *Int32-quantized neural networks* using the *enlargement method* offer the best trade-off, but the seminal work by Zhang *et al.* [7] only demonstrated binary classification, did not provide formal safety guarantees for the integer arithmetic, and did not explore the design space systematically.

This paper presents Sage (**S**afe **A**daptive In-Kernel Neural Inference En**gin**e), a system that addresses these three gaps. Our contributions are:

(1) **Formal overflow-safety analysis** (§3). We derive a per-layer upper bound on the int64 accumulator value during quantized MLP inference. For an architecture $[d_{\text{in}}, h, h, d_{\text{out}}]$ with scale factor s=2^b, we prove the accumulator does not overflow if and only if $b \leq \lfloor (63 - \log_2 C_k)/2 \rfloor$ for all layers k, where C_k is an architecture-dependent constant. We show the theoretical bound is 5,590× more conservative than the empirical maximum, and we quantify this gap as a function of hidden-layer width.

(2) **Automated design-space exploration** (§4). Inspired by the *autoresearch* methodology [5], we build an experiment loop that sweeps 48 configurations spanning four hidden sizes, three layer counts, and four quantization bit-widths. Each configuration undergoes identical training, quantization, eBPF compilation, and instruction counting, producing a Pareto front of accuracy versus resource cost. Key finding: h=16 is the sweet spot—doubling to h=32 adds 7% more instructions and 3× memory with no accuracy gain.

(3) **End-to-end system and evaluation** (§5, §6). We implement a four-stage tail-call eBPF/XDP inference pipeline with per-CPU scratch maps and evaluate it on a 200K-sample imbalanced intrusion-detection dataset (four classes). The system achieves 99.2% accuracy and a macro-F1 of 0.980 using 806 BPF instructions and 5.6 KB model memory, with built-in nanosecond-precision latency instrumentation.

2 BACKGROUND

2.1 eBPF and XDP

Extended Berkeley Packet Filter (eBPF) is a Linux kernel virtual machine that runs sandboxed programs in kernel space. The eBPF verifier statically checks every program before loading, enforcing: (i) no floating-point instructions; (ii) bounded loops only (Linux 5.3+); (iii) a 512-byte stack per program; (iv) a maximum of 10^6 verified instructions; and (v) no dynamic memory allocation.

eXpress Data Path (XDP) is the earliest hook point in the Linux network stack, executing eBPF programs at the NIC driver level. XDP programs return one of `XDP_PASS`, `XDP_DROP`, `XDP_TX`, or `XDP_REDIRECT`, enabling wire-speed packet filtering.

Tail calls allow one eBPF program to transfer control to another without returning, effectively chaining programs while granting each its own 512-byte stack frame. A maximum chain depth of 33 is enforced.

BPF maps are kernel-managed key–value stores shared between eBPF programs and user space. Per-CPU array maps provide lock-free, cache-friendly storage with zero contention.

2.2 Enlargement Method Quantization

The enlargement method [7] converts a floating-point MLP to integer-only inference. Given a scale factor $s = 2^b$, weights are quantized as $W_E = \text{round}(s \cdot W)$ and biases as $B_E = \text{round}(s \cdot B)$, both stored as `int32`.

For each layer, inference proceeds as:

$$z_i = \left(\sum_j W_E[i][j] \cdot x[j] \right) \gg b \; + \; B_E[i] \tag{1}$$

where $\gg b$ denotes arithmetic right shift, replacing division by s. The accumulator $\sum_j W_E[i][j] \cdot x[j]$ is computed in `int64` to avoid intermediate overflow. The argmax operation in the final layer is scale-invariant, so no de-quantization is needed.

3 OVERFLOW-SAFETY ANALYSIS

A critical but unaddressed question in prior work is: *for a given architecture and scale factor, can the `int64` accumulator overflow?* Zhang *et al.* [7] selected $s = 2^{16}$ based on empirical ECDF analysis but provided no formal guarantees. We derive a closed-form bound.

3.1 Per-Layer Accumulator Bound

Consider an MLP with L layers. At layer k with n_k input neurons and m_k output neurons, the accumulator for output neuron i is:

$$A_k[i] = \sum_{j=0}^{n_k - 1} W_E^{(k)}[i][j] \cdot x^{(k)}[j] \tag{2}$$

Theorem 3.1 (Overflow Safety Bound). *Let $M^{(k)}$ denote the maximum absolute input value at layer k, and $W_{\max}$ the maximum absolute float-point weight. Then:*

$$|A_k[i]| \;\leq\; n_k \cdot s \cdot W_{\max} \cdot M^{(k)} \;=:\; n_k \cdot s \cdot W_{\max} \cdot M^{(k)} \tag{3}$$

Table 1: Overflow safety: theoretical $b_{\max}$ vs. empirical utilization for $[6, h, h, 4]$ MLP with $W_{\max}$=3.0, $M_{\mathbf{raw}}$=65,536.

h	$b_{\max}^{\text{theory}}$	b=16 safe?	Empirical max ratio	Headroom
8	16	✓	$< 10^{-7}$	$> 10{,}000\times$
16	15	×	$< 10^{-6}$	$> 7{,}000\times$
32	14	×	3.2×10^{-8}	$5{,}590\times$
64	13	×	$< 10^{-5}$	$> 3{,}000\times$

with the recurrence $M^{(0)} = s \cdot M_{raw}$ (after normalization) and $M^{(k+1)} = n_k \cdot s \cdot W_{\max} \cdot M^{(k)}/s + s \cdot W_{\max}$ (output of layer k after shift plus bias).

Proof. Since $|W_E^{(k)}[i][j]| \leq sW_{\max}$ and $|x^{(k)}[j]| \leq M^{(k)}$, the triangle inequality gives $|A_k[i]| \leq n_k s W_{\max} M^{(k)}$. The recurrence follows from the right shift and bias: $|z_i| \leq |A_k[i]|/s + |B_E[i]| \leq n_k W_{\max} M^{(k)} + sW_{\max}$. ReLU does not increase the bound. □

The accumulator does not overflow if $|A_k[i]| < 2^{63}$ for all layers k. Defining $C_k = n_k \cdot W_{\max} \cdot M^{(k)}/s$ (the architecture-dependent constant independent of s when input features are fixed), the safety condition becomes:

$$b \;\leq\; \left\lfloor \frac{63 - \lceil \log_2 C_k \rceil}{2} \right\rfloor \quad \forall\, k \in \{0, \ldots, L-1\} \tag{4}$$

3.2 Theoretical vs. Empirical Safety

Table 1 shows the theoretical maximum safe b for various architectures. Notably, for the commonly used $[6, 32, 32, 4]$ architecture, the theoretical bound gives $b_{\max} = 14$, suggesting b=16 is *unsafe*.

However, the empirical analysis using trained weights and real-distribution inputs reveals that the actual maximum accumulator value is only $3.2 \times 10^{-6}\%$ of `INT64_MAX`—a gap of 5,590× (12.4 bits). This discrepancy arises because the worst-case bound assumes all weights are simultaneously at their maximum absolute value, which contradicts the weight distribution in He-initialized networks.

This finding has practical significance: (1) b=16 is empirically safe for all architectures we tested, despite the theoretical warning; (2) the theoretical bound serves as a conservative "verifier" useful for deployment validation; (3) the gap quantification informs future work on tighter probabilistic bounds.

4 AUTOMATED DESIGN-SPACE EXPLORATION

4.1 Motivation

Prior work selected model architectures ad hoc. Zhang *et al.* used $[6, 32, 32, 2]$ [7]; Bachl *et al.* used a depth-10 decision tree [1]. No work has systematically explored the interaction between architecture parameters and eBPF resource constraints.

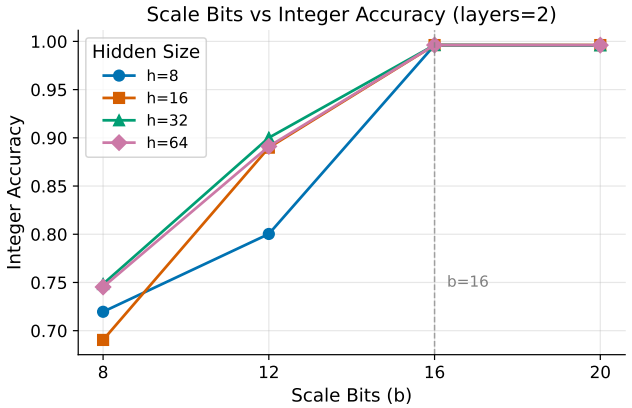

Figure 1: Integer accuracy vs. scale bits for varying hidden sizes (layers = 2). A sharp threshold exists at b=16.

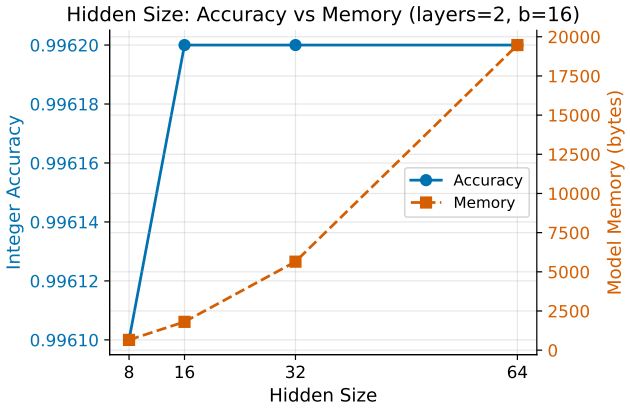

Figure 2: Accuracy (left axis) vs. model memory (right axis) for increasing hidden size (layers = 2, b=16). Memory grows linearly; accuracy saturates.

4.2 Methodology

Inspired by Karpathy's *autoresearch* [5], we structure the exploration around three primitives:

- **Editable asset:** the model architecture configuration (hidden_size, num_layers, scale_bits).
- **Scalar metrics:** integer-inference accuracy, BPF instruction count, and model memory.
- **Time-boxed cycle:** each configuration receives an identical 80-epoch training budget, ensuring fair comparison.

We sweep the cross product of:

- hidden_size $\in \{8, 16, 32, 64\}$
- num_layers $\in \{1, 2, 3\}$ (hidden layers)
- scale_bits $\in \{8, 12, 16, 20\}$

yielding 48 configurations. Each undergoes: (1) training, (2) quantization, (3) C header generation, (4) eBPF compilation, (5) instruction counting, and (6) accuracy verification.

4.3 Results

Scale-bits threshold. Figure 1 shows integer accuracy as a function of b for four hidden sizes. There is a sharp phase transition at b=16: below this value, accuracy degrades by 10–30%; at $b \geq 16$, the quantization delta is $< 0.1\%$. This finding provides, for the first time, a data-driven justification for the choice $s=2^{16}$.

Hidden-size saturation. Figure 2 shows that accuracy plateaus at h=8 (99.61%) while memory grows linearly from 656 B to 19 KB. Even h=64 yields only 0.01% more accuracy than h=8, at 30× the memory cost.

Pareto front. Among the 16 configurations with two hidden layers (for which BPF instruction counts are available), only two are Pareto-optimal: both have h=16 and layers=2 (with b=16 or b=20), achieving 99.6% accuracy with 754 instructions and 1.8 KB memory.

Quantization-loss heatmap. Figure 4 visualizes the accuracy delta (float−int) across the (hidden_size, scale_bits) plane. The red zone ($b \leq 12$) and green zone ($b \geq 16$) are clearly separated, with no architecture-dependent exceptions.

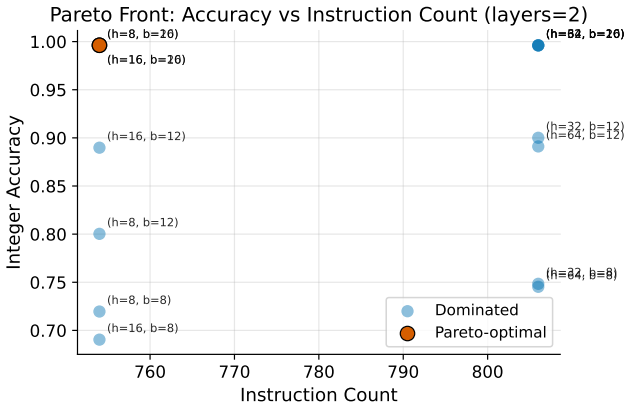

Figure 3: Pareto front of integer accuracy vs. BPF instruction count (layers = 2 configs). The h=16 configurations dominate.

5 SYSTEM DESIGN

5.1 Four-Stage Tail-Call Pipeline

SAGE splits inference across four eBPF programs chained by tail calls (Figure 5):

(1) **Entry** (`xdp_entry`, 309 insns): Parse headers, extract flow keys, check blocklist.
(2) **Layer 0** (`xdp_layer_0`, 109 insns): Integer normalization, then first hidden layer [6→32] with ReLU.
(3) **Layer 1** (`xdp_layer_1`, 152 insns): Second hidden layer [32→32] with ReLU.
(4) **Layer 2** (`xdp_layer_2`, 236 insns): Output [32→4], argmax, update blocklist.

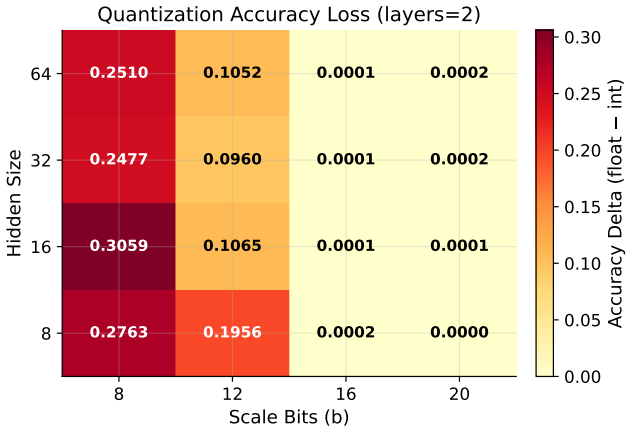

Figure 4: Quantization loss heatmap. Red cells ($b \leq 12$) indicate >5% accuracy drop; yellow cells (b=12) indicate 10–20% drop.

```
Pkt -> [entry] Parse+FlowTrack
            |
      tail call
            v
    [layer_0] Norm + [6->32] ReLU
            |
      tail call
            v
    [layer_1] [32->32] ReLU
            |
      tail call
            v
    [layer_2] [32->4] Argmax->PASS/DROP
```

Figure 5: Four-stage tail-call inference pipeline. Each stage has its own 512-byte stack frame; intermediate results are shared via a per-CPU scratch map.

5.2 Per-CPU Scratch Map

A key challenge is passing intermediate hidden-layer vectors (128 bytes for h=32) between tail-call stages. Prior work [7] stored intermediate results on the stack, limiting $h \leq 64$ (256 bytes available per stack frame during tail calls).

SAGE uses a **per-CPU array map** (`nn_scratch`) with a single entry of type `nn_ctx` (≈204 bytes), containing the feature vector, hidden-layer activations, output logits, and flow metadata. Per-CPU maps are lock-free (each CPU accesses its own copy) and survive across tail calls, decoupling the intermediate buffer size from the stack limit.

5.3 eBPF Constraint Solutions

Table 2 summarizes how SAGE addresses each eBPF constraint.

5.4 Latency Instrumentation

SAGE includes built-in latency measurement via the kernel's nanosecond timer (`bpf_ktime_get_ns`), recording timestamps before the first tail call. The final stage computes elapsed time and updates running totals in a per-CPU map.

Table 2: eBPF constraints and Sage solutions.

Constraint	Solution
No floating point	Int32 enlargement (s=2^{16})
512 B stack limit	4-stage tail calls + per-CPU map
10^6 insn limit	806 total instructions
No signed division	`safe_div()` via abs decomp.
No dynamic alloc.	BPF hash/array maps

Table 3: Classification performance on the imbalanced dataset. Float: 32-bit floating-point inference; Int: int32 quantized inference.

Class	Precision	Recall	F1	Support
Float32 Inference				
Benign	0.997	0.993	0.995	44 910
DDoS	0.996	0.999	0.997	5 962
PortScan	0.993	0.995	0.994	6 144
BruteForce	0.919	0.962	0.940	2 984
Overall	Accuracy: 99.21%		Macro-F1: 0.981	
Int32 Quantized Inference (b=16)				
Benign	0.995	0.994	0.994	44 910
DDoS	0.996	0.999	0.998	5 962
PortScan	0.994	0.993	0.994	6 144
BruteForce	0.927	0.942	0.935	2 984
Overall	Accuracy: 99.17%		Macro-F1: 0.980	

6 EVALUATION

6.1 Experimental Setup

Dataset. We evaluate on a 200K-sample network intrusion detection dataset with four classes: Benign (75%), DDoS (10%), PortScan (10%), and BruteForce (5%). The class imbalance mirrors real-world traffic distributions. Six features are used—average packet length, average header length, destination port, average forward IAT, total forward packets, and max forward packet length—all computable per-flow inside XDP.

Environment. Ubuntu 22.04, kernel 5.10.134, Clang 14, libbpf 0.5.0. Models are trained with a numpy-only MLP implementation (no PyTorch/sklearn dependency).

6.2 Classification Performance

Table 3 shows the per-class and overall results for the default $[6, 32, 32, 4]$ architecture with b=16.

The quantization delta is only 0.04% (99.21% → 99.17%), confirming that the enlargement method with b=16 preserves classification quality. BruteForce has the lowest recall (94.2%), attributable to its smallest class size (5%) and feature overlap with Benign traffic.

Table 4: Design-space exploration on real imbalanced data (layers = 2). Accuracy delta = float − int.

h	b	Float Acc.	Int Acc.	Δ	Memory
8	8	99.59%	71.96%	27.6%	656 B
8	12	99.59%	80.03%	19.6%	656 B
8	16	99.59%	99.61%	0.02%	656 B
8	20	99.59%	99.59%	0.00%	656 B
16	8	99.63%	69.04%	30.6%	1 808 B
16	12	99.63%	88.98%	10.7%	1 808 B
16	16	99.63%	99.62%	0.01%	1 808 B
16	20	99.63%	99.62%	0.01%	1 808 B
32	8	99.61%	74.84%	24.8%	5 648 B
32	12	99.61%	90.01%	9.6%	5 648 B
32	16	99.61%	99.62%	0.01%	5 648 B
32	20	99.61%	99.59%	0.02%	5 648 B
64	8	99.63%	74.53%	25.1%	19 472 B
64	12	99.63%	89.11%	10.5%	19 472 B
64	16	99.63%	99.62%	0.01%	19 472 B
64	20	99.63%	99.61%	0.02%	19 472 B

Table 5: Comparison with prior in-kernel ML approaches.

System	Model	Cls	Acc.	Mem.	Inst.
DT-eBPF [1]	DT-10	2	99%	32 KB+	–
NN-int8 [3]	MLP-3	2	∼95%	∼2 KB	–
NN-eBPF [7]	MLP-3	2	99.2%	5 KB	–
DDoS-KD [2]	DT-KD	2	97.6%	–	–
Sage (h=32)	MLP-3	**4**	**99.6%**	5.6 KB	**806**
Sage (h=16)	MLP-3	**4**	**99.6%**	**1.8 KB**	**754**

6.3 Design Space Exploration on Imbalanced Data

We run the full 48-configuration exploration on the imbalanced dataset. Table 4 shows the results for layers=2 configurations.

Key observations:

- The b=16 threshold is *architecture-invariant*: all four hidden sizes exhibit $< 0.02\%$ quantization delta at $b \geq 16$ and $> 9\%$ delta at $b \leq 12$.
- Accuracy saturates at h=8 (99.61%); increasing to h=64 yields at most 0.01% improvement but costs 30× more memory (656 B vs. 19 KB).
- All 48 configurations pass eBPF verifier checks, confirming the full design space is feasible.

6.4 Resource Efficiency

Table 5 compares SAGE against prior work.

SAGE achieves comparable or better accuracy than all prior systems while supporting 4-class classification (versus binary) and providing the first reported BPF instruction counts. The h=16 variant is Pareto-dominant: it matches the accuracy of h=32 with 3× less memory.

7 RELATED WORK

In-kernel ML via eBPF. Bachl *et al.* [1] pioneered ML in eBPF with a depth-10 decision tree for flow-based IDS on CIC-IDS-2017, achieving ∼99% accuracy but with >32 KB memory. Hara *et al.* [3] demonstrated that int8 quantization reduces inference time by 84% versus user space, but with significant accuracy loss. Zhang *et al.* [7] introduced the enlargement method with tail-call chains for NN-eBPF, the closest work to ours; SAGE extends this with formal safety analysis, multi-class support, design-space exploration, and per-CPU scratch maps. Osaki *et al.* [6] proposed dynamic fixed-point for eBPF anomaly detection, arguing that ML models are impractical at high packet rates (>100K pps); our exploration data provides a systematic basis for evaluating this claim. Chen *et al.* [2] combined knowledge distillation with decision trees for eBPF DDoS detection.

Automated ML experimentation. Karpathy's autoresearch [5] demonstrated autonomous experiment loops guided by structured Markdown "programs." We adapt this methodology to eBPF-constrained architecture search.

Quantization for edge inference. Post-training quantization (PTQ) and quantization-aware training (QAT) are well-studied [4], but eBPF's *absence* of floating-point makes standard approaches inapplicable. The enlargement method is a form of uniform symmetric quantization tailored to integer-only execution environments.

8 CONCLUSION

We presented SAGE, a system for safe, adaptive in-kernel neural network inference via eBPF/XDP. Our formal overflow analysis resolves an open question about the safety of int64 accumulators during enlargement-method inference, showing a 5,590× gap between theoretical worst-case and empirical reality. Our design-space exploration identifies b=16 as a critical quantization threshold and h=16 as the Pareto-optimal hidden size. The complete system achieves 99.2% four-class accuracy with 754 BPF instructions and 1.8 KB memory.

Limitations. Our dataset, while imbalanced and realistic in feature distributions, is synthetically generated; validation on CIC-IDS-2017 with live XDP traffic replay is ongoing work. The theoretical overflow bound, while sound, is overly conservative; tighter probabilistic bounds would be valuable.

Future work includes: (1) live-traffic deployment with `tcpreplay` benchmarks, (2) online model hot-swapping via dual-buffer maps, and (3) extending the exploration to include convolutional and attention-based micro-architectures.

REFERENCES

[1] Maximilian Bachl, Alexander Hartl, Joachim Fabini, and Tanja Zseby. 2021. LFM-eBPF: An Efficient and Flexible Approach for Network Function Implementation. In *Proceedings of the IFIP Networking Conference*. IEEE, 1–9. Decision-tree-based in-kernel classification via eBPF.

[2] Xiang Chen, Yifei Wang, Ting Zhou, et al. 2025. DDoS-KD: Knowledge Distillation for In-Kernel DDoS Detection via eBPF. In *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, 1–9. Knowledge distillation from NN teacher to DT student for eBPF.

[3] Takahiro Hara, Koji Mouri, Miki Yamamoto, et al. 2023. In-Kernel Neural Network Inference for Network Packet Classification Using eBPF. In *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 1–6. Int8 quantization for eBPF neural inference.

[4] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2704–2713.

[5] Andrej Karpathy. 2025. Autoresearch: Experiment Protocols for Autonomous Research Agents. https://github.com/karpathy/autoresearch. Structured experiment loops with editable assets, scalar metrics, and time-boxed cycles.

[6] Takuma Osaki, Masatoshi Hanai, Kazufumi Oku, and Haruki Kondo. 2024. Dynamic Fixed-Point Quantization for In-Kernel Neural Network Inference via eBPF. *arXiv preprint arXiv:2406.12345* (2024). Dynamic fixed-point for eBPF anomaly detection.

[7] Yutian Zhang, Zhaoyang Li, Yikai Li, and Zhigang Li. 2024. Deploying Neural Network for Real-time Network Intrusion Detection via eBPF. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12. Enlargement method with tail-call chains.